Introduction to Operating System and System calls

1. Motivation (Why do we need an OS?)

Consider the program:

```java
int[] arr = new int[100];
for(int i = 0; i < 100; i++) {
   System.out.println(arr[i]);
}
```

At first glance, this looks simple. But internally, many operations are happening:

- Memory is allocated
- Instructions are executed
- Output is displayed on screen

Key Question:
Who manages all of this?

Answer: The Operating System (OS)

---

2. What is an Operating System?

An **Operating System (OS)** is **system software that acts as an interface between the user and the computer hardware, and manages all resources of the system.**

In simpler terms:
- It is the **boss of the computer** that makes sure everything runs smoothly.

Examples of Operating Systems:

- Windows
- Linux
- macOS
- Android

What is common across all OS?

They manage:

- Memory
- Programs
- Devices
- User interaction

---

3. Analogy

Component	Real World
User	Customer
Program	Order
OS	Waiter
Hardware	Kitchen

You (user) do not interact with hardware directly.
The OS (waiter) takes your request and gets it done.

---

4. How Your Program Actually Runs

4.1 Memory Management

> Memory Management is the function of the OS that **allocates, tracks, and manages memory (RAM)** for all running programs.

In simple words:

- Who gets memory?
- How much memory?
- Where in memory?
- When to free it?

All of this is handled by the OS.

When you write:

```java
new int[100]
```

What happens:

1. Program (via JVM) requests memory
2. OS checks available RAM
3. OS allocates a block of memory
4. OS marks it as used
5. OS returns starting address

Key Concept:

- Each program gets **its own memory space**
- Programs cannot access each other's memory

4.2 Process Management

A **program** becomes a **process** when it starts executing

> Process = Program in execution
>
> Your Java code → when you click **Run** → becomes a **process**

Process Management is another **core function of the Operating System**. It is responsible for **running programs efficiently by managing processes**. It is the function of the OS that **creates, schedules, executes, and terminates processes**.

In simple terms:

- Which program should run?
- For how long?
- When should it stop?

All decisions are taken by the OS.

OS Responsibilities:

- Create process
- Maintain list of processes
- Allocate CPU time

Examples of processes:

- Running Java program
- Browser
- Background system tasks

4.3 CPU Execution

CPU Execution explains **how your program's instructions are actually run** and how the **Operating System ensures fair and efficient use of the CPU**.

It is the process where the **CPU runs instructions of a program step by step**, while the OS manages *which process gets CPU time and when*.

For loop execution:

```java
for(int i = 0; i < 100; i++)
```

What happens:

- Your program becomes a **process**
- OS gives it access to the CPU
- CPU executes instructions **one by one**
- OS may pause it and give CPU to another process
- Then resumes it again
- OS ensures:
    - Fair CPU allocation
    - Multiple programs run smoothly

Role of OS in CPU Execution

The CPU does the execution, but the OS controls **who gets to use it**.

CPU Scheduling

- Many processes are ready to run
- OS decides:
    - Which process runs first
    - How long it runs

4.4 System Calls

Programs cannot directly access hardware.

They use **System Calls**.

Definition:

> System calls are the interface through which a program requests services from the OS.

Examples:

- `new int[100]` → Memory request
- `System.out.println()` → Output request
- File creation → Disk operation

Why Do We Need System Calls?

A program cannot:

- Access memory directly
- Write to disk directly
- Control devices directly

Why?

- Security
- Stability
- Controlled access

So instead, programs say:

> "OS, please do this for me"

Key Idea:

Program → System Call → OS → Hardware

4.5 Device Management

Device Management is the function of the OS that **handles communication between programs and hardware devices**.

Examples of devices:

- Keyboard
- Mouse
- Screen (monitor)
- Printer
- Disk (HDD/SSD)

Example: Printing output

Programs **do not know how hardware works internally**.

For example:

- A Java program cannot directly control a screen or printer
- Hardware has complex low-level operations

So the OS acts as a **middle layer**.

Steps:

1. Program requests output
2. OS receives request
3. OS uses **device driver**
4. Driver communicates with hardware
5. Output appears

Note: A **device driver** is software that helps the OS communicate with a specific hardware device.

Key Idea:

- Each device has its own driver
- OS uses drivers as translators

Example:

- Printer driver
- Display driver
- Keyboard driver

4.6 File Management

When saving a file:

- OS decides storage location
- Uses file system to track data
- Retrieves file when needed

4.7 Security

Security is a critical function of the OS that ensures the system remains **safe, reliable, and protected from unauthorized access or misuse**.

In a system with multiple users and programs:

- Programs may try to access others' data
- Malicious software may attempt harmful actions
- Accidental errors may corrupt data

The OS prevents all of this.

OS ensures:

- Programs are isolated
- Permissions are controlled
- Unauthorized access is prevented
- **Confidentiality** → Data is accessed only by authorized users
- **Integrity** → Data is not modified incorrectly
- **Availability** → Resources are available when needed

---

5. Key Visualization

```None
User Program → Operating System → Hardware
                      ↑
                System Calls
```

---

6. What Happens Without OS?

Without an OS:

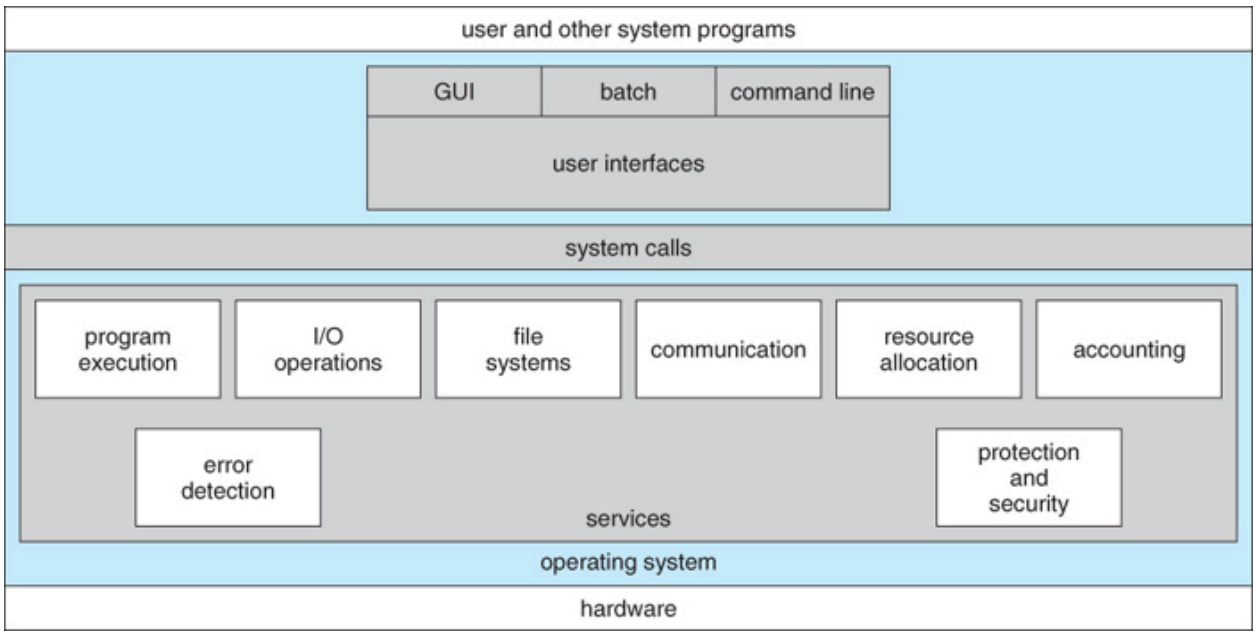
- No memory allocation
- No CPU execution
- No file handling
- No output display

Conclusion:

With out OS, your code is just text.

---

7. Functions of Operating System



7.1 User Interface (UI)

The User Interface is how users **interact with the Operating System**.

Types of Interfaces:

- **CLI (Command Line Interface)**
  - Users type commands (e.g., `ls`, `cd`, `javac`)
  - Requires remembering commands
  - Fast and powerful
- **GUI (Graphical User Interface)**

    - Uses windows, icons, buttons, mouse
    - Example: Desktop, file explorer
    - Easy to use
- **Batch Interface**
    - Commands are written in a file and executed together
    - No user interaction during execution

Key Idea:

OS provides different ways for users to communicate with the system.

7.2 Program Execution

OS is responsible for **running programs from start to finish**.

Steps involved:

1. Load program into memory
2. Start execution using CPU
3. Monitor execution
4. Handle termination:
    - **Normal** → Program finishes successfully
    - **Abnormal** → Crash or error

Key Idea:

Without OS, programs cannot start or run.

7.3 I/O Operations

OS manages all **input/output operations**.

Examples:

- Reading a file
- Writing to a file
- Taking keyboard input
- Displaying output on screen

Why OS is needed:

- Direct device access is unsafe
- OS provides controlled access using system calls

Key Idea:

Programs request → OS performs I/O safely

7.4 File System Management

OS manages how data is **stored and organized on disk**.

Responsibilities:

- Create/delete files
- Read/write file data
- Organize files in directories
- Manage permissions (who can access what)

Key Idea:

OS acts like a **file organizer** for the system.

7.5 Communication (Inter-Process Communication - IPC)

OS allows processes to **communicate and share data**.

Methods:

- **Shared Memory**
  - Multiple processes access same memory
  - Fast but needs synchronization
- **Message Passing**
  - Processes send messages via OS
  - Safer and more controlled

Example:

- Browser process talking to network process

7.6 Error Detection

OS continuously checks for **errors in the system**.

Types of errors:

- **Hardware errors** → CPU, memory issues
- **Memory errors** → invalid access
- **Program errors** → divide by zero, crash

OS Action:

- Detect error
- Take corrective action
- Prevent system failure

Key Idea:

OS keeps system stable and reliable.

7.7 Resource Allocation

Resource allocation is one of the **core responsibilities of an OS**.
The idea is simple:

- **Many programs want resources at the same time, but resources are limited.**
- The OS decides **who gets what, when, and for how long.**

What are “Resources”?

Resources are anything a program needs to run:

- **CPU (processor time)**
- **Memory (RAM)**
- **I/O devices** (keyboard, mouse, printer, disk)
- **Files & storage**

How OS helps:

- CPU scheduling (who runs next)
- Memory allocation (who gets how much)
- Device allocation (who uses device)

Key Idea:

OS ensures **fair and efficient usage**

7.8 Accounting

Accounting in an OS is the process of tracking and recording resource usage by users and processes for analysis, billing, and optimization.

Tracks:

- CPU time used
- Memory usage
- Disk usage

Why Accounting is Important

The OS doesn't just allocate resources—it also keeps a **record of usage** to:

- Analyze performance
- Ensure fair usage
- Control costs (especially in shared systems)
- Detect misuse or overuse

Key Idea:

OS maintains usage records.

7.9 Protection & Security

Protection & Security in Operating System

These two terms are related but different:

- **Protection** → Controls how processes use resources inside the system
- **Security** → Protects the system from unauthorized users and external threats

---

1. Protection (Internal Control)

Protection ensures that one program does not interfere with another or with the OS.

Why it is needed

- Multiple programs run at the same time
- Without protection, one program could corrupt another's data

How OS provides protection

Memory Protection

- Each process gets its own memory space
- One program cannot access another program's memory

File Protection

- Controls permissions: read, write, execute

CPU Protection

- Ensures no program uses the CPU forever
- Uses time sharing (time quantum)

User Mode vs Kernel Mode

- User Mode → Limited access
- Kernel Mode → Full access (OS runs here)

---

2. Security (External Protection)

Security focuses on protecting the system from unauthorized access.

How OS provides security

Authentication

- Verifies user identity (username, password)

Authorization

- Controls what a user can access

Encryption

- Protects data by making it unreadable

Network Security

- Protects against hackers and malware

Key Difference

Protection	Security
Internal control	External defense
Between processes	Against attackers
Controls resource access	Protects whole system

- **Protection** = Rules inside a college (who can access which room)
- **Security** = Guards at the gate (who can enter the campus)

Final Summary

All these functions together make the OS:

> A **resource manager + control system + security layer**

When any program runs:

- UI helps interaction
- OS executes program

- Handles I/O
- Manages files
- Enables communication
- Detects errors
- Allocates resources
- Tracks usage
- Ensures security

---

8. System Calls

System Calls are **controlled interfaces that allow a user-level program to request specific services (like file access, process creation, or I/O operations) from the operating** system, causing a safe switch from user mode to hardware mode for execution. System calls are the bridge between user programs and the OS.

Why Needed?

- Programs cannot access hardware directly
- OS ensures safety and efficiency

---

9. System Call Demo

STEP 1 — Create Java Program

nano SyscallDemo.java

STEP 2 — Paste Code

Paste this inside nano:

```
import java.io.FileOutputStream;
public class SyscallDemo {
    public static void main(String[] args) throws Exception {
```

```
        FileOutputStream fos = new FileOutputStream("demo.txt");
        fos.write("Hello OS\n".getBytes());
        fos.close();
        System.out.println("Done");
    }
}
```

STEP 3 — Save and Exit (Very Important)

1. Press Control + O
2. Press Enter
3. Press Control + X

STEP 4 — Confirm File Exists

ls

You should see: SyscallDemo.java

STEP 5 — Compile Program

javac SyscallDemo.java

No output means success.

STEP 6 — Run Program

java SyscallDemo

Output: Done

STEP 7 — Check File Created

cat demo.txt

Output: Hello OS

What to Say “So our program works. It created a file and wrote data.”

“But who actually did this work?”

Pause.

System Call Demo

STEP 8 — Open Terminal 1 (Monitoring)

sudo fs_usage -w | grep demo.txt

- `sudo`
  Runs the command with admin permission (needed to see system-level activity)
- `fs_usage`
  macOS tool that shows **file system activity in real time**
- `-w`
  Displays output in a readable (wide) format
- `| grep demo.txt`
  Filters output to show only lines related to **demo.txt**

“Shows live Operating System activity only for our file `demo.txt`.”

It will ask for password.
Type your Mac password (nothing will be visible while typing) and press Enter.

STEP 9 — Open Terminal 2 (Run Program)

java SyscallDemo

STEP 10 — Observe Output (Terminal 1)

You will see lines containing:

- demo.txt
- WrData (means writing)
- file access activity

This is the Operating System working on our file.

Lecture 2 - Processes and Context Switching

Processes, CPU Scheduling, and Context Switching

---

1. Introduction

Modern systems run multiple applications simultaneously:

- Browser (Chrome)
- Code Editor (VS Code)
- Music Player (Spotify)
- Terminal

Key Questions

- How does one CPU handle multiple applications?
- Does each application get its own processor?
- How does the system switch between applications?
- How does it resume execution correctly?

Core Idea

The Operating System (OS) enables multitasking using:

- **Processes**
- **CPU Scheduling**
- **Context Switching**
- **Efficient Memory Management**

---

2. Process

Definition

A **Process** is an instance of a program in execution, along with its current state and resources.

Program vs Process

Program	Process
Static (stored on disk)	Dynamic (in execution)

Passive entity	Active entity
No state	Has state (PC, registers, memory)

Components of a Process

Each process consists of:

- **Text Section** → Executable code
- **Data Section** → Global/static variables
- **Heap** → Dynamically allocated memory
- **Stack** → Function calls, local variables
- **Program Counter (PC)** → Address of next instruction
- **CPU Registers** → Temporary storage for execution

Process Control Block (PCB)

The OS maintains a data structure called the **Process Control Block (PCB)** for each process.

PCB contains:

- Process ID (PID)
- Process state
- Program Counter
- CPU registers
- Memory management info
- Scheduling information (priority, queue pointers)
- I/O status information

---

3. CPU Architecture Reality

Single-Core CPU

- Executes **one instruction at a time**
- No true parallelism

Multi-Core CPU

- Multiple cores → **true parallel execution**
- Each core executes one instruction stream independently

Key Insight

Even with multiple cores:

- Number of processes >> number of cores
- OS still needs **scheduling**

---

4. CPU Scheduling

Definition

CPU Scheduling is the mechanism by which the OS selects a process from the **ready queue** and allocates CPU time.

Scheduling Queue Types

- **Ready Queue** → Processes waiting for CPU
- **Waiting Queue** → Processes waiting for I/O

Scheduling Goals

- **CPU Utilization** → Maximize CPU usage
- **Throughput** → Increase number of completed processes
- **Turnaround Time** → Minimize total execution time
- **Waiting Time** → Reduce idle waiting
- **Response Time** → Fast user interaction

Common Scheduling Algorithms (Overview)

- First Come First Serve (FCFS)
- Shortest Job First (SJF)
- Round Robin (RR)
- Priority Scheduling

---

5. Time Slice (Quantum)

Definition

Time Quantum (or Time Slice) is the **fixed, predefined interval of CPU time** assigned to a process in preemptive scheduling, after which the operating system forcibly interrupts (preempts) the process, saves its execution context, and allocates the CPU to another process from the ready queue to ensure fair and efficient sharing of the CPU.

Round Robin Scheduling (Will be discussed in detail in the next class)

- Each process gets CPU for a fixed quantum
- After quantum expires → **Process leaved the Core**

Execution Example

```
None
A → B → C → A → B → C ...
```

Each process runs for a short duration.

Key Insight

- Creates **concurrency illusion**
- Improves responsiveness
- Prevents starvation (in many cases)

---

6. Process States

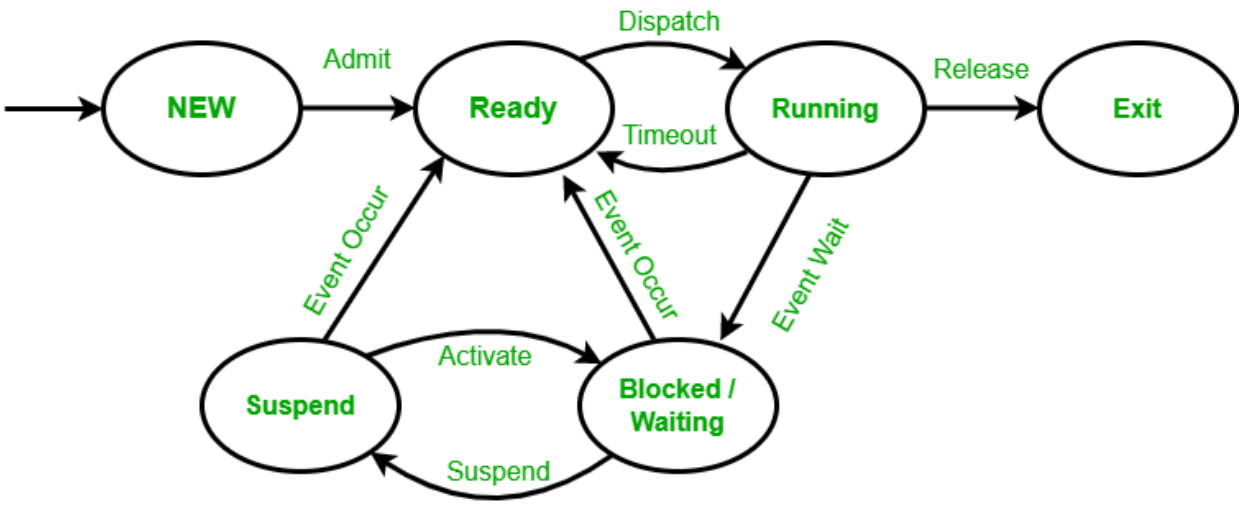
Basic States

- **New** → Process is being created
- **Ready** → Waiting for CPU

- **Running** → Executing
- **Waiting (Blocked)** → Waiting for I/O
- **Terminated** → Execution finished

State Transitions

- Ready → Running (CPU allocated)
- Running → Waiting (I/O request)
- Waiting → Ready (I/O complete)
- Running → Ready (preemption)



---

7. Context Switching

Definition

Context Switching is the mechanism of saving the execution state of a currently running process and restoring the state of another process.

What is Context?

The **execution context** includes:

- Program Counter (PC)
- CPU Registers (general-purpose + special registers)

- Stack Pointer
- Process state
- Memory mapping (page tables, base/limit registers)

---

8. Context Switch: Internal Steps

Step-by-Step

1. Interrupt or scheduler decision occurs
2. Save current process context into PCB
3. Update process state (Running → Ready/Waiting)
4. Select next process from ready queue
5. Load context of selected process from PCB
6. Update state (Ready → Running)
7. Resume execution

---

9. Role of Interrupts

Context switching is triggered by:

- **Timer Interrupt** (time quantum expiry)
- **I/O Interrupt** (I/O completion)
- **System Calls**
- **Hardware Interrupts**

---

11. Context Switching Overhead

Important Concept

Context switching introduces **overhead** because:

- No useful work is done during switching
- CPU spends time saving/restoring state

Types of Costs

- Register save/restore time
- Cache invalidation (possible)
- Pipeline stalls
- Memory access delays

Performance Impact

- Frequent switching → reduced CPU efficiency
- Large quantum → poor responsiveness
- Small quantum → high overhead

Summary

- Multitasking is mostly an **illusion created by fast switching**
- A process is more than code — it includes state and resources
- Context switching is fundamental to OS design
- Efficient scheduling balances performance and fairness